Class-Uniform Codes in Every Characteristic but Three, Their Blow-ups, and the Price of Advice

Hans Brende Claude Opus 5.5 (Anthropic) Gemini 4 Argon (Google DeepMind)

November 2026

Abstract

The parent paper studies class-uniform rules: row c of a $d \times d$ matrix is a fixed linear function of the matrix with its columns rotated by c, the function depending only on the class of c in a partition of $\mathbb{Z}/d$. Its Theorem TP bounds the codes of such a rule by $(d^2 + Md)/2$ when the M classes are intervals, and its Proposition 8.2 shows that the bound can fail for classes with no almost-period: at $d = 6$ the classes $\{0, 1, 3\}$ and $\{2, 4, 5\}$ carry a code of dimension $25 > 24$ over $\mathbb{F}_2$. This memo adds three things.

First, this gain does not come from characteristic 2: the same classes carry a code of dimension 25 over every field in which $3 \neq 0$, in particular over $\mathbb{Q}$, $\mathbb{R}$ and $\mathbb{C}$.

Second, blowing up a class-uniform code by a factor k multiplies its dimension by k^2 and its number of classes by k, so it multiplies the code's excess over the interval value by k^2. Applied to the code at $d = 6$ it gives, for $d = 6k$ and $2k$ classes, codes whose excess is k^2. This answers the parent paper's Problem 9.1(c) as worded, but only because the number of classes grows: the excess is $dM/12$, of the order of the interval term $dM/2$, and blow-ups leave the ratio of the excess to dM unchanged. We state the problem for a bounded number of classes.

Third, we price advice in the one-hop model. Advice that is a linear map of rank q costs at most q dimensions, however each row uses it, and every code extends by exactly q dimensions with suitable advice of rank q. Over a finite field, advice of any kind with q symbols costs at most q dimensions plus half a row for each class change it induces. The parent paper suggests charging each bit revealed by a depth reduction as a regime change, at half a row. In the one-hop model the advice itself costs one dimension per symbol, and only the class changes it induces cost half a row each.

Every theorem, proposition and corollary is proved in Lean, on top of the parent paper's library; one remark records computations that the data generator checks. All results concern the linear one-hop model of the parent paper, not machines.

Contributions. Hans Brende chose the questions and directed the research. Gemini 4 Argon, a large language model developed by Google DeepMind, found the two rational codes of Section 3 by computer search, gave the first Lean proof of the code over the fields in which $1008 \neq 0$, proposed the blow-up of Section 4 in a coset form, and proved, in related one-hop models, that linear advice of rank q costs at most q dimensions. Claude Opus 5.5, a large language model developed by Anthropic, checked these results, identified the blow-up as a tensor construction and worked out what it says about the parent paper's problems, proved the remaining results of Section 5, wrote the Lean library of this memo, and wrote the text. This memo extends the parent paper [1], which, with its formalization, is unchanged.

Contents

1 Introduction **2**

1.1 Results . . . 2
1.2 How to read this memo . . . 3
1.3 Organization . . . 3

2 The model **4**

3 Every characteristic but three **4**

4 Blow-ups **6**

5 The price of advice **7**

6 What remains **9**

A Index of Lean declarations **10**

B Building and verifying the formalization **11**

1 Introduction

The series asks whether a multitape Turing machine can transpose a $d \times d$ matrix of $b(d)$-bit entries in time linear in its length $L = d^2 b(d)$; this is statement (T) of the first paper [2]. The parent paper [1] isolates a linear one-hop model, which it reads as what one pass over a tape can decode, and proves two rigidity theorems in it. One of them, Theorem TP [1, Theorem 6.4], concerns *class-uniform rules*: row c of a matrix X is a fixed linear function of X with its columns rotated by c, and the function depends only on the class of c in a partition of $\mathbb{Z}/d$ [1, Definition 6.3]. If the M classes are cyclic intervals, every code of such a rule has dimension at most $(d^2 + Md)/2$: half the matrix space, plus half a row for each interval. For classes with no almost-period the bound can fail: at $d = 6$ the classes $\{0, 1, 3\}$ and $\{2, 4, 5\}$ carry a code of dimension $25 > 24$ over $\mathbb{F}_2$ [1, Proposition 8.2]. The parent paper asks how large such gains can be [1, Problem 9.1], and lists three steps that separate Theorem TP from a statement about oblivious machines [1, Section 7.4]. The second, depth reduction, needs a version of Theorem TP in which values revealed at chosen steps of the computation enter as advice.

1.1 Results

Every characteristic but three. The code of dimension 25 is not an artefact of characteristic 2 (Theorem 3.2): for the classes $\{0, 1, 3\}$ and $\{2, 4, 5\}$ of $\mathbb{Z}/6$ there is a code of dimension 25 over every field F in which $3 \neq 0$. The proof transfers three explicit certificates along ring homomorphisms (Proposition 3.1): two codes over $\mathbb{Q}$, whose denominators involve the primes $2, 3, 7$ and $2, 3, 5, 13$ respectively, and the parent paper's code over $\mathbb{F}_2$. So the gain of the parent paper's Proposition 8.2 exists over $\mathbb{Q}$, $\mathbb{R}$ and $\mathbb{C}$. It is the one gain over the interval value that the parent paper proves; its table of small sizes also reports gains for three classes, numerically and over $\mathbb{F}_2$. Characteristic 3 is open (Problem 6.1).

Blow-ups. Write a point of $\mathbb{Z}/(dk)$ as $c = ka + u$ with $a \in \mathbb{Z}/d$ and $0 \leq u < k$. The blow-up of a code $\mathcal{X}$ of $d \times d$ matrices consists of the $dk \times dk$ matrices all of whose k^2 blocks $\left(X_{ka+u,\, kt+v}\right)_{a,t}$ lie in $\mathcal{X}$. It is a tensor product $\mathcal{X} \otimes \mathrm{M}_k(F)$ up to a reindexing. Theorem 4.2 shows that the blow-up of a class-uniform code is class-uniform for the colouring $c \mapsto$ (class of a, u), which has k times as

many classes, and that its dimension is $k^2 \dim \mathcal{X}$. So the excess $\dim \mathcal{X} - (d^2 + Md)/2$ of a code over the interval value is multiplied by k^2, and so is dM (Corollary 4.3). Applied to the code at $d = 6$, this gives for every k a code of dimension $25k^2$ for $2k$ classes on $\mathbb{Z}/6k$, with excess k^2 (Corollary 4.4). That answers Problem 9.1(c) of the parent paper as worded, which asks for a family, with a description that can be analysed, in which the excess grows; the family can be analysed given its base code, which a solver found. We read (c), with the parent paper's Section 7.3 and Remark 8.1(d), as asking whether the gain stays bounded at a fixed number of classes, and the family does not touch that: its excess is $dM/12$, the same order as the term $dM/2$ that intervals already give, and the ratio of the excess to dM is invariant under blow-ups. The two merged classes form four intervals, and for $k \geq 2$ Theorem TP excludes the same dimension for them (Proposition 4.5). We restate the problem for a bounded number of classes (Problem 6.2).

The price of advice. A class-uniform rule with linear advice α lets row c depend, besides the class-uniform term, on $\alpha(X)$ through a linear map that may differ from row to row. The advice costs at most its rank (Theorem 5.2): the matrices of the code on which α vanishes form a code of a class-uniform rule for the same colouring. And every code extends by exactly q dimensions with suitable advice of rank q, for every $q \leq d^2 - \dim \mathcal{X}$ (Proposition 5.3). Over a finite field the advice may be of any kind (Theorem 5.5): an arbitrary map to a finite set Ω, on each of whose fibres the rows obey a class-uniform rule, with offsets, for a colouring that may depend on the advice value. Then $|F|^{\dim \mathcal{X}} \leq |\Omega| \cdot |F|^D$, where D bounds the codes of the colourings that the advice selects. With Theorem TP, if the class changes of every selected colouring lie in a non-empty set of at most s rows, this gives $2 \dim \mathcal{X} \leq d^2 + ds + 2q$ for q symbols of advice. So each symbol of advice costs one dimension, and each class change it induces costs half a row. The parent paper suggests charging each revealed bit as a regime change, at half a row, and proves nothing about it [1, Section 7.4]. In the one-hop model that charge is pessimistic: advice that enters linearly induces no class change and costs one dimension per symbol. Remark 5.6 says what remains for machines.

1.2 How to read this memo

Every theorem, proposition, corollary and definition carries a status line in the style of the parent paper. `Lean`: names the declarations and the file of the formal proof, and an *outline of the formal proof* follows; the Lean statement is authoritative. Declarations are relative to the namespace `ClassUniformMemo` unless they carry the prefix `SwapRigidity`, the parent paper's namespace; the files are in `lean/ClassUniformMemo/`. Every theorem, proposition and corollary is formalized, except for arithmetic restatements inside a statement, such as the form of a bound for M intervals, which follow from the formal statement at once. Remark 3.3 records facts about the data that the data generator checks and that no proof uses. The two results of the parent paper that we use are formalized there. The Lean library builds on the parent paper's library, which it requires through a relative path, and uses no axioms beyond `propext`, `Classical.choice` and `Quot.sound` (Appendix B). The Lean statements index rows and columns by `ZMod n`; the text writes d for the side length, as the parent paper does.

All results are about the linear one-hop model. None of them is a statement about machines, and none of them changes the status of (T).

1.3 Organization

Section 2 recalls the model. Section 3 proves the code of dimension 25 in every characteristic but three, Section 4 constructs the blow-ups, and Section 5 prices advice. Section 6 states what remains.

Appendix A indexes the Lean declarations, and Appendix B says how to build and check them.

2 The model

Let F be a field and $d \geq 1$. Rows and columns of a matrix $X \in \mathrm{M}_d(F)$ are indexed by $\mathbb{Z}/d$, and $x_c \in F^{\mathbb{Z}/d}$ is row c. The rotation σ^c moves the columns: $(\sigma^c X)_{i,j} = X_{i,j+c}$ (`SwapRigidity.rot`). We use the parent paper's definitions.

Definition 2.1 (Class-uniform rules [1, Definition 6.3])**.** Lean: `SwapRigidity.rot`, `SwapRigidity.IsClassUniformCode` [`ClassUniform.lean`]

Fix a colouring of $\mathbb{Z}/d$, that is, a partition $\mathcal{P}$ into classes. A *class-uniform rule* is a family of linear maps $\mathcal{A}_J \colon \mathrm{M}_d(F) \to F^{\mathbb{Z}/d}$, one for each class J. A subspace $\mathcal{X} \subseteq \mathrm{M}_d(F)$ is a *code* of the rule if $x_c = \mathcal{A}_J(\sigma^c X)$ for every $X \in \mathcal{X}$, every class J and every $c \in J$. We write $D(\mathcal{P})$ for the largest dimension of a code of a class-uniform rule for $\mathcal{P}$, over the field at hand.

The file named in the status line is the parent paper's `ClassUniform.lean`. In Lean the colouring is a map `cls` from $\mathbb{Z}/d$ to a type of classes, and `SwapRigidity.IsClassUniformCode` asserts that decoders exist.

Theorem 2.2 (Theorem TP [1, Theorem 6.4])**.** Cited: the parent paper, Theorem 6.4, Lean-verified there (`SwapRigidity.IsClassUniformCode.two_mul_finrank_le`); we use its form for the unit shift, as in that Lean statement.

Let $\mathcal{X}$ be a code of a class-uniform rule, and let $S \subseteq \mathbb{Z}/d$ be a non-empty set that contains every c whose class differs from that of $c-1$. Then $2\dim \mathcal{X} \leq d^2 + d|S|$. In particular, if the classes are $M \geq 2$ cyclic intervals, then $\dim \mathcal{X} \leq (d^2 + Md)/2$.

For the parent paper a partition into M classes has an *almost-period* if a single shift leaves only M components in the graph of its Theorem 6.4; intervals have the almost-period 1. The classes $\{0,1,3\}$ and $\{2,4,5\}$ of $\mathbb{Z}/6$ have none: every nonzero shift leaves four components [1, Remark 8.1(c)].

Proposition 2.3 ([1, Proposition 8.2])**.** Cited: the parent paper, Proposition 8.2, Lean-verified there (`SwapRigidity.isClassUniformCode_aperiodicCode`, `SwapRigidity.finrank_aperiodicCode`, `SwapRigidity.aperiodicCls_eq_true_iff`).

Over $\mathbb{F}_2$ the classes $\{0,1,3\}$ and $\{2,4,5\}$ of $\mathbb{Z}/6$ carry a code of a class-uniform rule of dimension 25, while $(d^2 + Md)/2 = 24$.

The parent paper asks [1, Problem 9.1]: (a) whether $D(\mathcal{P}) \leq (\frac{1}{2} + o(1))d^2$ for two classes, or for $o(d)$ classes; (b) the value of D for these two classes over $\mathbb{F}_2$, which is between 25 and 28; and (c) for a family of partitions and codes, with a description that can be analysed, in which the *excess* $D(\mathcal{P}) - (d^2 + Md)/2$ grows. We call $\dim \mathcal{X} - (d^2 + Md)/2$ the excess of a code $\mathcal{X}$; the excess of a partition is at least that of each of its codes.

3 Every characteristic but three

A code is described by finite data over a commutative ring, and the data are moved to a field by a ring homomorphism.

Proposition 3.1 (Certificates)**.** Lean: `Certificate`, `Certificate.RowsHold`, `Certificate.PivotsHold`, `Certificate.code`, `Certificate.dec`, `Certificate.row_eq_dec_of_mem`, `Certificate.isClassUniformCode`, `Certificate.finrank_code` [`Transfer.lean`]

Let R be a commutative ring and $\varphi\colon R \to F$ a ring homomorphism to a field. Fix a colouring of $\mathbb{Z}/d$, matrices $G_1, \dots, G_m \in \mathrm{M}_d(R)$, for every class J and every $r \in \mathbb{Z}/d$ coefficients $a_{J,r}(i,j) \in R$, elements $N, P \in R$, and positions $p_1, \dots, p_m$, such that

(i) $N\,(G_k)_{c,r} = \sum_{i,j} a_{J,r}(i,j)\,(G_k)_{i,j+c}$ *for every k, every r, and every c in a class J;*

(ii) $(G_k)_{p_l} = P$ *if $k = l$ and* $(G_k)_{p_l} = 0$ *otherwise.*

If $\varphi(N) \neq 0$, the span of the matrices $\varphi(G_k)$ is a code of the class-uniform rule $\mathcal{A}_J(Y)_r = \varphi(N)^{-1} \sum_{i,j} \varphi(a_{J,r}(i,j))\, Y_{i,j}$. If moreover $\varphi(P) \neq 0$, it has dimension m.

Outline of the formal proof. Applying φ to (i) and dividing by $\varphi(N)$ shows that each $\varphi(G_k)$ satisfies the rule (`Certificate.genMap_row`); the rule is linear, so the span satisfies it (`Certificate.row_eq_dec_of_mem`). The map $Y \mapsto (\varphi(P)^{-1} Y_{p_l})_l$ sends $\varphi(G_k)$ to the k-th standard basis vector, so the $\varphi(G_k)$ are linearly independent (`Certificate.linearIndependent_genMap`). □

Theorem 3.2 (Theorem A, every characteristic but three)**.** Lean: `exists_code_of_three_ne_zero`, `exists_classUniformCode_gt_of_three_ne_zero`, `exists_code_of_charZero`, `exists_code_zmod` [`Fields.lean`]

Let F be a field in which $3 \neq 0$. Then the classes $\{0,1,3\}$ and $\{2,4,5\}$ of $\mathbb{Z}/6$ carry a code of a class-uniform rule of dimension 25 over F, so that $2 \dim \mathcal{X} = 50 > 48 = 6^2 + 2 \cdot 6$. In particular this holds over every field of characteristic zero and over $\mathbb{F}_p$ for every prime $p \neq 3$.

Outline of the formal proof. Three certificates in the sense of Proposition 3.1, all for the parent paper's colouring `SwapRigidity.aperiodicCls`, cover the fields.

- `rationalA`, over $R = \mathbb{Z}$ with φ the canonical map, has $N = 1008 = 2^4 \cdot 3^2 \cdot 7$ and $P = 3$. It serves every field in which 2, 3 and 7 are nonzero.
- `rationalB`, over $\mathbb{Z}$, has $N = 187200 = 2^6 \cdot 3^2 \cdot 5^2 \cdot 13$ and $P = 15$. In characteristic 7 its scales are $187200 = 6$ and $15 = 1$, both nonzero; it serves characteristic 7.
- `binary`, over $R = \mathbb{F}_2$, is the parent paper's code with $N = P = 1$ (`binary_rowsHold`, `binary_pivotsHold`); with φ the map $\mathbb{F}_2 \to F$ it serves characteristic 2.

The conditions (i) and (ii) for the two rational certificates, 900 identities between integers and 625 entries each, are proved by `decide` (`rationalA_rowsHold`, `rationalA_pivotsHold`, `rationalB_rowsHold`, `rationalB_pivotsHold`). The proof of `exists_code_of_three_ne_zero` splits on whether $2 = 0$ in F, then on whether $7 = 0$; in the remaining case $1008 = 2^4 \cdot 3^2 \cdot 7 \neq 0$. For $\mathbb{F}_p$, $3 = 0$ only when p divides 3. □

Remark 3.3 (Numerical: the certificates)**.** The data are in `tools/data/rational_a.json` and `tools/data/rational_b.json`, each a class-uniform rule over $\mathbb{Q}$. The generator `tools/gen_rational_data.py` brings the 36×36 system of the rule to reduced echelon form, takes the standard basis of its solution space (equal to 1 at one free position and 0 at the others), clears the denominators of the basis and of the decoders, and writes the integer data to `RationalData.lean`. Lean does not trust the generator: the kernel checks the identities (i) and (ii), and the proofs use nothing else. Two further facts are checked by exact computation only, by the generator and by an independent recomputation, and no proof uses them: the solution space of each rule has dimension exactly

25, so the span of the generators is the whole code of the rule, and the two codes are different subspaces of $\mathrm{M}_6(\mathbb{Q})$, meeting in dimension 20. Between them the two rational certificates serve every characteristic except 2 and 3.

The parent paper records that on cone relations, linear recovery with bounded complex coefficients and bounded width stays at the pairing rate, so that a linear construction needs positive characteristic or coefficients that cannot be rescaled to be bounded [1, Section 4.5]. A reader could take the gain of Proposition 2.3, the one gain over the interval value that the parent paper proves, for a phenomenon of characteristic 2 of the same kind. Theorem 3.2 shows that it is not. It concerns a single size and says nothing about rates, so it does not bear on that statement; it removes the only reason the examples gave to restrict a search for larger gains to positive characteristic. Characteristic 3 is open (Problem 6.1).

4 Blow-ups

Fix $k \geq 1$. Every $c \in \mathbb{Z}/dk$ is $c = ka + u$ for a unique *block* $a = \lfloor c/k \rfloor \in \mathbb{Z}/d$ and *position* $u = c \bmod k \in \{0, \dots, k-1\}$, where c is read as an integer in $[0, dk)$.

Definition 4.1 (Blocks and blow-ups)**.** `Lean`: `emb`, `hi`, `lo`, `block`, `blowUp`, `blowCls` [`BlowUp.lean`]

For $X \in \mathrm{M}_{dk}(F)$ and positions u, v, the *block* $X^{(u,v)} \in \mathrm{M}_d(F)$ is $X^{(u,v)}_{a,t} = X_{ka+u,\, kt+v}$. The *blow-up* of a subspace $\mathcal{X} \subseteq \mathrm{M}_d(F)$ is

$$\mathcal{X}^{(k)} = \{X \in \mathrm{M}_{dk}(F) : X^{(u,v)} \in \mathcal{X} \text{ for all positions } u, v\}.$$

The *blow-up* of a colouring of $\mathbb{Z}/d$ is the colouring of $\mathbb{Z}/dk$ that gives $c = ka + u$ the class of a together with the position u.

Up to the reindexing $c \leftrightarrow (a, u)$ of rows and of columns, $\mathcal{X}^{(k)}$ is the tensor product $\mathcal{X} \otimes \mathrm{M}_k(F)$. A colouring with M classes blows up to one with kM classes.

Theorem 4.2 (Theorem B, blow-ups)**.** `Lean`: `isClassUniformCode_blowUp`, `finrank_blowUp`, `blowDec`, `readBlock`, `rot_block`, `blowUpEquiv` [`BlowUp.lean`]

If $\mathcal{X}$ is a code of a class-uniform rule for a colouring of $\mathbb{Z}/d$, then $\mathcal{X}^{(k)}$ is a code of a class-uniform rule for the blown-up colouring of $\mathbb{Z}/dk$, and $\dim \mathcal{X}^{(k)} = k^2 \dim \mathcal{X}$.

Outline of the formal proof. Let $c = ka' + u$. Rotating the columns of X by c moves column $kt + v$ to column $kt + v - c$, and $(kt + v) - u + (ka' + u) = k(t + a') + v$ in $\mathbb{Z}/dk$ (`emb_sub_add_emb`). Hence the block (u, v) of X, rotated by the block a' of c, is read from $\sigma^c X$ in place: $(\sigma^{a'} X^{(u,v)})_{a,t} = (\sigma^c X)_{ka+u,\, kt+v-u}$ (`rot_block`; the right side is `readBlock`). This depends on u but not on a', so the decoder of the class (J, u) can compute it: it applies $\mathcal{A}_J$ to these blocks of $\sigma^c X$, one for each v (`blowDec`). Since $X^{(u,v)} \in \mathcal{X}$, row a' of $X^{(u,v)}$, which is the part of row c of X in the columns at position v, equals $\mathcal{A}_J(\sigma^{a'} X^{(u,v)})$ (`isClassUniformCode_blowUp`). For the dimension, $X \mapsto (X^{(u,v)})_{u,v}$ is an isomorphism of $\mathcal{X}^{(k)}$ onto $\mathcal{X}^{k \times k}$ (`blowUpEquiv`, `finrank_blowUp`). □

Corollary 4.3 (The excess scales)**.** `Lean`: `two_mul_finrank_blowUp_sub` [`BlowUp.lean`]

For every subspace $\mathcal{X} \subseteq \mathrm{M}_d(F)$ and every M,

$$2 \dim \mathcal{X}^{(k)} - \big((dk)^2 + (dk)(kM)\big) = k^2\big(2 \dim \mathcal{X} - (d^2 + dM)\big),$$

with subtraction truncated at zero, as in Lean. So blowing up multiplies the excess over the interval value by k^2. It also multiplies dM by k^2, since $(dk)(kM) = k^2 dM$: the ratio of the excess to dM is unchanged.

Outline of the formal proof. Substitute $\dim \mathcal{X}^{(k)} = k^2 \dim \mathcal{X}$ and distribute k^2 over the truncated difference. □

Corollary 4.4 (A family with growing excess)**.** Lean: `exists_blowUp_code`, `two_mul_finrank_blowUp_code`, `blowCls_surjective`, `card_blowClasses` [`Excess.lean`]

Let F be a field in which $3 \neq 0$, and $k \geq 1$. The blow-up of the colouring $\{0,1,3\},\{2,4,5\}$ of $\mathbb{Z}/6$ is a colouring of $\mathbb{Z}/6k$ with $2k$ classes, and it carries a code of a class-uniform rule of dimension $25k^2$. So

$$2\dim \mathcal{X} = (6k)^2 + (6k)(2k) + 2k^2,$$

and the excess over $(d^2+Md)/2 = 24k^2$ is $k^2 = dM/12$, for $d = 6k$ and $M = 2k$.

Outline of the formal proof. Blow up the code of Theorem 3.2 with Theorem 4.2 (`exists_blowUp_code`), and count (`two_mul_finrank_blowUp_code`). The blown-up colouring takes all $2k$ values (`blowCls_surjective`, `card_blowClasses`). □

Proposition 4.5 (The decoders cannot ignore the position)**.** Lean: `mergedCls`, `mergedChanges`, `mergedCls_sub_one`, `two_mul_finrank_le_merged`, `finrank_lt_of_mergedCls` [`Excess.lean`]

Merging the classes (J,u) of Corollary 4.4 over u gives the colouring $c \mapsto J(\lfloor c/k \rfloor)$ of $\mathbb{Z}/6k$, with two classes. Every code of a class-uniform rule for it has $2\dim \mathcal{X} \leq (6k)^2 + 6k \cdot 4$, and so, for $k \geq 2$, dimension less than $25k^2$.

Outline of the formal proof. The merged colouring changes class only at 0, $2k$, $3k$ and $4k$, the first rows of the blocks $0,2,3,4$ (`mergedChanges`, `mergedCls_sub_one`): inside a block the class is constant, and from block 1 to block 0 and from block 5 to block 4 it does not change (`hi_sub_one_of_lo_ne_zero`, `hi_sub_one_of_lo_eq_zero`). Theorem 2.2 with these four rows gives the bound (`two_mul_finrank_le_merged`), and $36k^2+24k < 50k^2$ for $k \geq 2$ (`finrank_lt_of_mergedCls`). □

So for $k \geq 2$ the codes of Corollary 4.4 are not codes of any rule for the two merged classes: their decoders cannot ignore the position u.

Corollary 4.4 answers Problem 9.1(c) of the parent paper as worded: it is a family of partitions and codes, with a description that can be analysed given its base code (which a solver found), in which the excess grows. It answers it only because the number of classes grows with d. The excess of these codes, $dM/12$, is of the same order as the term $dM/2$ that intervals give, and Corollary 4.3 shows that blow-ups cannot change the ratio of the two: a single code with excess e at (d,M) yields codes with excess ek^2 at (dk,kM), and nothing at a fixed number of classes. Problem 9.1(a) is untouched. A negative answer to it needs an excess of order d^2 with $M = o(d)$ classes, so the ratio of the excess to dM must be unbounded, which no blow-up of a finite example provides. We read Problem 9.1(c), with the parent paper's Section 7.3 and Remark 8.1(d), as asking whether the gain stays bounded at a fixed number of classes; Problem 6.2 states that.

5 The price of advice

The parent paper's second step from the one-hop model to machines is depth reduction [1, Section 7.4]: a genie reveals values at chosen steps, and the revealed word is advice that depends on position, so it breaks uniformity wherever it is used. The parent paper suggests that a version of Theorem TP with advice would charge each revealed bit as a regime change, at half a row per change, and proves nothing about it. This section prices advice in the one-hop model: at most one dimension per symbol, and exactly that for suitable advice, when it enters linearly; and over a finite field at most one dimension per symbol plus half a row for each class change it induces.

Definition 5.1 (Linear advice)**.** Lean: `IsClassUniformCodeWithAdvice` [`Advice.lean`]

Let $\alpha\colon \mathrm{M}_d(F) \to W$ be a linear map to any vector space. A subspace $\mathcal{X}$ is a *code of a class-uniform rule with linear advice* α if there are decoders $\mathcal{A}_J$, one for each class, and linear maps $B_c\colon W \to F^{\mathbb{Z}/d}$, one for each row, such that $x_c = \mathcal{A}_J(\sigma^c X) + B_c(\alpha(X))$ for every $X \in \mathcal{X}$, every class J and every $c \in J$.

The maps B_c may differ from row to row, so each row may use the advice in its own way.

Theorem 5.2 (Theorem C, linear advice costs its rank)**.** Lean: `IsClassUniformCodeWithAdvice.inf_ker`, `finrank_le_finrank_inf_ker_add`, `IsClassUniformCodeWithAdvice.two_mul_finrank_le` [`Advice.lean`]

Let $\mathcal{X}$ be a code of a class-uniform rule with linear advice α. Then $\mathcal{X} \cap \ker\alpha$ is a code of a class-uniform rule for the same colouring, and $\dim\mathcal{X} \le \dim(\mathcal{X} \cap \ker\alpha) + \operatorname{rank}\alpha$*. Consequently, if S is as in Theorem 2.2, then* $2\dim\mathcal{X} \le d^2 + d|S| + 2\operatorname{rank}\alpha$*; for $M \ge 2$ interval classes,* $\dim\mathcal{X} \le (d^2 + Md)/2 + \operatorname{rank}\alpha$*.*

Outline of the formal proof. On $\mathcal{X} \cap \ker\alpha$ the advice term vanishes, so the same decoders work (`IsClassUniformCodeWithAdvice.inf_ker`). By rank and nullity for the restriction of α to $\mathcal{X}$, with values in the range of α, the kernel of the restriction, which is $\mathcal{X} \cap \ker\alpha$, has codimension at most $\operatorname{rank}\alpha$ in $\mathcal{X}$ (`finrank_le_finrank_inf_ker_add`). The last statement adds Theorem 2.2 (`IsClassUniformCodeWithAdvice.two_mul_finrank_le`). □

Proposition 5.3 (The price is exact)**.** Lean: `isClassUniformCodeWithAdvice_sup_range`, `finrank_sup_range` [`Advice.lean`]

Let $\mathcal{X}_0$ be a code of a class-uniform rule, and p a linear projection of $\mathrm{M}_d(F)$ ($p^2 = p$) that vanishes on $\mathcal{X}_0$. Then $\mathcal{X}_0 + \operatorname{im}p$ is a code of a class-uniform rule for the same colouring, with linear advice p, and $\dim(\mathcal{X}_0 + \operatorname{im}p) = \dim\mathcal{X}_0 + \operatorname{rank}p$*.*

Outline of the formal proof. Take $B_c(w) = w_c - \mathcal{A}_J(\sigma^c w)$ for $c \in J$. For $X = x + p(z)$ with $x \in \mathcal{X}_0$ we have $p(X) = p(z)$, and $\mathcal{A}_J(\sigma^c X) + B_c(p(X)) = x_c + p(z)_c = X_c$ (`isClassUniformCodeWithAdvice_sup_range`). The sum is direct, because $y \in \mathcal{X}_0 \cap \operatorname{im}p$ gives $y = p(y) = 0$ (`finrank_sup_range`). □

A projection of any rank $q \le d^2 - \dim\mathcal{X}_0$ vanishing on $\mathcal{X}_0$ exists (project onto a q-dimensional subspace that meets $\mathcal{X}_0$ only in 0, along a complement containing $\mathcal{X}_0$). So every code of every class-uniform rule extends, with linear advice of rank q, by exactly q dimensions, for every $q \le d^2 - \dim\mathcal{X}_0$: the inequality $\dim\mathcal{X} \le \dim(\mathcal{X} \cap \ker\alpha) + \operatorname{rank}\alpha$ of Theorem 5.2 is sharp for every code. Its form with Theorem 2.2 is attained only from codes that attain Theorem 2.2.

Advice that is not linear may also change the decoders of the rows that use it. Over a finite field we can price that as well.

Definition 5.4 (Advice of any kind)**.** Lean: `IsCodeWithAnyAdvice`, `ruleSpace` [`Advice.lean`]

Let Ω be a finite set, $w\colon \mathrm{M}_d(F) \to \Omega$ any map, and $\mathcal{P}_\omega$, for $\omega \in \Omega$, colourings of $\mathbb{Z}/d$. A subspace $\mathcal{X}$ is a *code with advice w* for the colourings $\mathcal{P}_\omega$ if for every ω there are decoders $\mathcal{A}^\omega_J$, one for each class J of $\mathcal{P}_\omega$, and offsets $b^\omega_c \in F^{\mathbb{Z}/d}$, one for each row, such that $x_c = \mathcal{A}^\omega_J(\sigma^c X) + b^\omega_c$ for every $X \in \mathcal{X}$ with $w(X) = \omega$, every class J of $\mathcal{P}_\omega$ and every $c \in J$.

The offsets let the advice enter every row in its own way, and the colouring $\mathcal{P}_\omega$ lets it change the decoders of the rows that use it. A code of a class-uniform rule with linear advice α of rank q, over a finite field, is a code with advice $w = \alpha$, $|\Omega| = |F|^q$ after restricting the values to the range of α, and $\mathcal{P}_\omega = \mathcal{P}$ for all ω.

Theorem 5.5 (Theorem D, the price of advice)**.** Lean: `IsCodeWithAnyAdvice.natCard_pow_finrank_le`, `IsCodeWithAnyAdvice.natCard_pow_two_mul_finrank_le`, `natCard_submodule` [`Advice.lean`]

Let F be a finite field and $\mathcal{X}$ a code with advice $w\colon \mathrm{M}_d(F) \to \Omega$ for colourings $\mathcal{P}_\omega$.

(i) If every code of a class-uniform rule for every $\mathcal{P}_\omega$ has dimension at most D, then $|F|^{\dim \mathcal{X}} \le |\Omega| \cdot |F|^D$.

(ii) If for every ω some non-empty set of at most s rows contains every row whose class under $\mathcal{P}_\omega$ differs from that of the row before it, then $|F|^{2\dim \mathcal{X}} \le |\Omega|^2 \cdot |F|^{d^2+ds}$.

In particular, with $|\Omega| = |F|^q$, that is, q symbols of advice, $2\dim \mathcal{X} \le d^2 + ds + 2q$.

Outline of the formal proof. Fix ω and a matrix X_ω of $\mathcal{X}$ with $w(X_\omega) = \omega$, if there is one. For every other such X, the difference $X - X_\omega$ lies in $\mathcal{X}$ and satisfies the rule of ω without offsets, so it lies in the code $\mathcal{X} \cap \{Y : y_c = \mathcal{A}^\omega_J(\sigma^c Y)\}$ (`ruleSpace`), which has at most $|F|^D$ elements (`natCard_submodule`). The fibres of w partition $\mathcal{X}$, so $|\mathcal{X}| \le |\Omega| \cdot |F|^D$ (`IsCodeWithAnyAdvice.natCard_pow_finrank_le`). For (ii), Theorem 2.2 gives $D = \lfloor (d^2 + ds)/2 \rfloor$; square (`IsCodeWithAnyAdvice.natCard_pow_two_mul_finrank_le`). □

Remark 5.6 (What this says about depth reduction)**.** Theorem 5.5 is a version of Theorem 2.2 with advice, in the one-hop model over a finite field. Every symbol of advice costs one dimension, and the advice costs more only through the class changes it induces, at half a row each. The parent paper suggests charging each revealed bit as a regime change, at half a row. Advice that enters linearly, as it does when the revealed values are linear in the input and enter the rows additively, induces no class change, and its price is one dimension per symbol (Theorem 5.2 and Proposition 5.3). Advice that changes the decoder of a single row inside an interval induces two class changes, a full row. In the regime of depth reduction, $q \approx \varepsilon d^2$ symbols, part (ii) is informative only if $s < d(1 - 2\varepsilon)$: the revealed values may induce fewer than d class changes in all. The count agrees with the parent paper's charge for a state of q symbols carried across a regime change, of order qd when each of the d output rows carries its own state [1, Section 7.2].

For machines the step remains open. Theorem 5.2 uses that $\mathcal{X} \cap \ker \alpha$ is a code, and the proof of Theorem 5.5 cancels the offsets by subtracting a matrix of the fibre, so that each fibre lies in a coset of a code. With nonlinear decoders an offset that depends on the row is itself a break of uniformity. So the second step of [1, Section 7.4] needs, for machines, a version of Theorem 2.2 for the cone systems of real schedules, whose decoders are compositions of the transition function and need not be linear; a form of it that tolerates offsets depending on the row; and a bound on the class changes that a genie induces.

6 What remains

Two questions remain open after this memo: one about the field, and the parent paper's Problem 9.1(c) in a sharper form.

Problem 6.1 (Characteristic three)**.** *Over a field of characteristic* 3*, do the classes* $\{0, 1, 3\}$ *and* $\{2, 4, 5\}$ *of* $\mathbb{Z}/6$ *carry a code of a class-uniform rule of dimension* 25*?*

Both rational certificates of Theorem 3.2 have scales divisible by 3 ($N = 1008$ and $P = 3$; $N = 187200$ and $P = 15$), so neither certificate reduces modulo 3. A positive answer would make the gain of Proposition 2.3 independent of the field; a negative one would make it depend on the characteristic.

Problem 6.2 (Problem 9.1(c) of the parent paper, for a bounded number of classes)**.** *Fix M, for instance $M = 2$. Is the excess $D(\mathcal{P}) - (d^2 + Md)/2$ bounded over all d and all partitions $\mathcal{P}$ of $\mathbb{Z}/d$ into M classes? If not, find a family of such partitions and codes, with a description that can be analysed, in which it grows.*

By Corollary 4.3, blow-ups trade a growing number of classes for a growing excess at a fixed ratio, so they do not bear on Problem 6.2. A growing excess at a fixed number of classes would be a coding gain that grows with d and is native to masks without an almost-period, the first of its kind in the one-hop model; it would still be consistent with a positive answer to Problem 9.1(a), which concerns gains of order d^2.

The steps from the one-hop model to machines [1, Section 7.4] change as follows. Depth reduction, the second step, is priced in the one-hop model over finite fields by Theorem 5.5; for machines it still needs what Remark 5.6 lists. The first step, a version of Theorem 2.2 for the cone systems of real schedules, and the third, many hops, are unchanged. Statement (T) remains open.

A Index of Lean declarations

Every declaration of this memo's library cited in the text, grouped by the Lean module that contains it, relative to the namespace `ClassUniformMemo`. All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix B). The declarations of the parent paper's library named in the text are checked both in that library and in `PaperAxioms.lean`: `SwapRigidity.IsClassUniformCode`, `SwapRigidity.IsClassUniformCode.two_mul_finrank_le`, `SwapRigidity.aperiodicCls`, `SwapRigidity.aperiodicCls_eq_true_iff`, `SwapRigidity.finrank_aperiodicCode`, `SwapRigidity.isClassUniformCode_aperiodicCode`, `SwapRigidity.rot` in the parent paper's library `SwapRigidity` [1].

`ClassUniformMemo.Advice` `IsClassUniformCodeWithAdvice`, `IsClassUniformCodeWithAdvice.inf_ker`, `IsClassUniformCodeWithAdvice.two_mul_finrank_le`, `IsCodeWithAnyAdvice`, `IsCodeWithAnyAdvice.natCard_pow_finrank_le`, `IsCodeWithAnyAdvice.natCard_pow_two_mul_finrank_le`, `finrank_le_finrank_inf_ker_add`, `finrank_sup_range`, `isClassUniformCodeWithAdvice_sup_range`, `natCard_submodule`, `ruleSpace`.

`ClassUniformMemo.BlowUp` `block`, `blowCls`, `blowDec`, `blowUp`, `blowUpEquiv`, `emb`, `emb_sub_add_emb`, `finrank_blowUp`, `hi`, `hi_sub_one_of_lo_eq_zero`, `hi_sub_one_of_lo_ne_zero`, `isClassUniformCode_blowUp`, `lo`, `readBlock`, `rot_block`, `two_mul_finrank_blowUp_sub`.

`ClassUniformMemo.Excess` `blowCls_surjective`, `card_blowClasses`, `exists_blowUp_code`, `finrank_lt_of_mergedCls`, `mergedChanges`, `mergedCls`, `mergedCls_sub_one`, `two_mul_finrank_blowUp_code`, `two_mul_finrank_le_merged`.

`ClassUniformMemo.Fields` `binary`, `binary_pivotsHold`, `binary_rowsHold`, `exists_classUniformCode_gt_of_three_ne_zero`, `exists_code_of_charZero`, `exists_code_of_three_ne_zero`, `exists_code_zmod`, `rationalA`, `rationalA_pivotsHold`, `rationalA_rowsHold`, `rationalB`, `rationalB_pivotsHold`, `rationalB_rowsHold`.

`ClassUniformMemo.Transfer` `Certificate`, `Certificate.PivotsHold`, `Certificate.RowsHold`, `Certificate.code`, `Certificate.dec`, `Certificate.finrank_code`, `Certificate.genMap_row`, `Certificate.isClassUniformCode`, `Certificate.linearIndependent_genMap`, `Certificate.row_eq_dec_of_mem`.

B Building and verifying the formalization

The Lean project is the directory `lean/` of this memo. It uses Lean and Mathlib, both at version v4.34.1 (`lean-toolchain`, `lakefile.toml`, `lake-manifest.json`), and requires the parent paper's library `SwapRigidity` through the relative path `../../lean`; that library requires the shared `lean-machine/` library of the repository, which supplies Mathlib at the pinned revision. Keep the repository's directory layout so that both stay in place.

Build. With `elan` installed, run in `lean/`:

```
lake exe cache get && lake build
```

The first command downloads the compiled Mathlib cache; the second compiles `ClassUniformMemo` and, if needed, the parent paper's library.

Axioms and index. The file `lean/PaperAxioms.lean`, generated from the LaTeX sources of this memo by `tools/lean_index.py`, runs `#print axioms` on every declaration cited in this memo (Appendix A):

```
lake env lean PaperAxioms.lean
```

Every declaration depends at most on `propext`, `Classical.choice` and `Quot.sound`. No Lean file of this library uses `sorry`, `admit` or `native_decide`, declares an `axiom`, bypasses the kernel via `unsafe`, `implemented_by` or `extern`, or changes `maxHeartbeats` or `maxRecDepth`. Running `python3 tools/lean_index.py` from the memo's root directory regenerates both `paper/sections/leanindex.tex` and `lean/PaperAxioms.lean` (`-check` verifies that every cited name of this memo's library resolves and that both generated files are current; the names of the parent paper's library are resolved when `PaperAxioms.lean` runs).

Where things are. The library `lean/ClassUniformMemo/` has the following modules, all imported by the root module `lean/ClassUniformMemo.lean`.

- `Transfer.lean` (Proposition 3.1): certificates over a commutative ring and their transfer to fields.
- `RationalData.lean`: the integer data of the two rational codes, generated by `tools/gen_rational_data.py` from `tools/data/rational_a.json` and `tools/data/rational_b.json`. It contains definitions only; run the generator with `-check` to confirm that it is current.
- `Fields.lean` (Theorem 3.2): the three certificates, the finite checks by `decide`, which the kernel checks, and the theorem for every field in which $3 \neq 0$.
- `BlowUp.lean` (Theorem 4.2 and Corollary 4.3): blocks, blow-ups, the decoders of a blow-up, and its dimension.
- `Excess.lean` (Corollary 4.4 and Proposition 4.5): the family on $\mathbb{Z}/6k$, and the bound for the two merged classes.
- `Advice.lean` (Section 5): linear advice and its exact price, and advice of any kind over a finite field.

Computations. The only computation is the generation of `RationalData.lean` (Remark 3.3). The generator needs only the Python standard library. Before it writes the data it re-checks them, and it also checks the two facts of Remark 3.3 that Lean does not: that the solution space of each rule has dimension 25, and that the two codes span a space of dimension 30. No proof depends on a computation that Lean does not check.

References

[1] Hans Brende and Claude Fable 5.2 (Anthropic). Order, symmetry and the swap: A synthesis of matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision b2c389a2eea0fcd3654cd3ff8ba652a6d98ba547. URL: https://github.com/HansBrende/papers/tree/b2c389a2eea0fcd3654cd3ff8ba652a6d98ba547/Order-Symmetry-and-the-Swap-A-Synthesis-of-Matrix-Transposition-on-Tapes.

[2] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 4e402bdcd3e123d529dc160b992fe27e92152449. URL: https://github.com/HansBrende/papers/tree/4e402bdcd3e123d529dc160b992fe27e92152449/Matrix-Transposition-and-the-Tape-Coding-Conjecture.